DataMetaMap: A Library for Dataset Vector Representation

Vladislav Minashkin, Ivan Papay, Vladislav Meshkov, Ilya Stepanov
Intelligent Systems
MIPT 2025

CONTENTS

LIST OF FIGURES

DataMetaMap: A Library for Dataset Vector Representation

***Abstract*—This report presents *DataMetaMap*, a Python library designed to represent multiple datasets in a unified vector space, enabling principled comparison of dataset similarity. The library offers a comprehensive suite of dataset embedding techniques compatible with PyTorch, addressing the challenge of transferring knowledge across datasets with varying schemas in meta-learning pipelines. We study four complementary approaches: Maximum Mean Discrepancy (MMD), Task2Vec, Dataset2Vec, and Wasserstein Task Embedding. The repository includes executable demos and benchmark pipelines for dataset similarity analysis, transfer-oriented retrieval, and warm-start hyperparameter optimization across heterogeneous datasets.**

I. Introduction

A fundamental challenge in modern machine learning is the efficient transfer of knowledge across tasks and datasets. Given a new target dataset, a practitioner must decide which pretrained model or which prior learning experience is most relevant — a decision that currently relies heavily on human intuition or exhaustive search.

A principled solution requires the ability to *compare* datasets quantitatively: if datasets can be embedded into a common vector space where geometric proximity reflects task similarity, then finding the most relevant prior experience reduces to a nearest-neighbor query. Such embeddings, often called *meta-features* or *task embeddings*, form the foundation of data-driven meta-learning.

Existing approaches to dataset representation span a wide spectrum. Classical engineered meta-features (number of instances, class imbalance, statistical moments of features) are cheap to compute but have limited expressivity and require schema-specific design. Kernel-based methods such as Maximum Mean Discrepancy (MMD) provide distribution-level comparisons without parametric assumptions but scale poorly with dimensionality. More recent neural and geometry-aware approaches — Task2Vec, Dataset2Vec, and Wasserstein Task Embedding — learn rich, task-agnostic representations but differ substantially in their inductive biases, computational cost, and applicability to datasets with varying schemas.

We present `DataMetaMap`, a unified Python library that combines these dataset embedding methods within a practical, PyTorch-compatible framework. Our contributions are:

- A unified overview of four families of dataset comparison methods: MMD, Task2Vec, Dataset2Vec, and Wasserstein Task Embedding.
- A clean, extensible repository packaging Dataset2Vec and Wasserstein embedders together with a Task2Vec subpackage, shared utilities, and benchmark code.
- Experimental benchmarks for transfer-oriented dataset retrieval and warm-start hyperparameter optimization.
- Method-specific demo notebooks illustrating training, visualization, and similarity analysis in the learned embedding spaces.

II. Notation and Conventions

We adopt the following notation throughout this report.

a) Datasets and meta-datasets.: A dataset is denoted $D = (X, Y)$, where $X \in \mathbb{R}^{N\times M}$ is the predictor matrix and $Y \in \mathbb{R}^{N\times T}$ is the target matrix; N, M, and T are the number of instances, predictors, and targets, respectively. A *meta-dataset* is a collection $\mathcal{D}_{\text{meta}} = \{D_1, \ldots, D_K\}$ of (possibly heterogeneous) datasets used for meta-training or evaluation.

b) Meta-features.: A *meta-feature extractor* is a map $\varphi : \mathcal{D} \to \mathbb{R}^d$ that assigns to every dataset a fixed-dimensional embedding $\varphi(D) \in \mathbb{R}^d$. When the extractor is itself a learned (estimated) object, we write $\hat{\varphi}$. For methods based on stochastic batch sampling, D_s and D'_s denote multi-fidelity subsets (batches) drawn from D by subsampling instances, predictors, and targets.

c) Probe networks.: For Task2Vec, we additionally introduce a *probe network* with parameters $\theta \in \mathbb{R}^P$ that is briefly fine-tuned on the target dataset. The Fisher Information Matrix (FIM) of the probe parameters is denoted $F(\theta) \in \mathbb{R}^{P\times P}$.

d) Distributional and kernel quantities.: We write $k(\cdot, \cdot)$ for a positive-definite kernel (e.g. RBF), μ_P, μ_Q for kernel mean embeddings of distributions P and Q in a reproducing kernel Hilbert space (RKHS) $\mathcal{H}$, and $W_p(P, Q)$ for the p-Wasserstein distance. In the Wasserstein Task Embedding section, ψ denotes the MDS map used to embed class-label distributions into a Euclidean space.

e) Hyperparameters.: Throughout, $\gamma > 0$ denotes a bandwidth or scaling hyperparameter for similarity models, and $\tau > 0$ a temperature parameter where applicable. The binary indicator $i \in \{0, 1\}$ marks whether two batches originate from the same dataset ($i = 1$) or not ($i = 0$).

III. Proposed Solutions

A. Maximum Mean Discrepancy (MMD)

Overview: Gretton et al. [1] introduced the Maximum Mean Discrepancy as a non-parametric, kernel-based measure of the distance between two probability distributions. Originally proposed as a principled two-sample test, MMD operates by embedding distributions into a reproducing kernel Hilbert space (RKHS) and measuring the distance between their mean embeddings. The result is a scalar dissimilarity score that can serve directly as a dataset distance measure, without requiring an explicit parametric model.

Methodology: Given two distributions P and Q with samples $\{x_i\}_{i=1}^m \sim P$ and $\{y_j\}_{j=1}^n \sim Q$, the squared MMD in an RKHS $\mathcal{H}$ induced by kernel k is

$$\mathrm{MMD}^2(P,Q) = \big\|\mu_P - \mu_Q\big\|_{\mathcal{H}}^2, \tag{1}$$

where $\mu_P = \mathbb{E}_{x\sim P}[k(x,\cdot)]$ is the kernel mean embedding of P. An unbiased empirical estimator is

$$\begin{aligned}\widehat{\mathrm{MMD}}^2(P,Q) = &\frac{1}{m(m-1)}\sum_{i\neq i'} k(x_i,x_{i'}) \\ &+ \frac{1}{n(n-1)}\sum_{j\neq j'} k(y_j,y_{j'}) \\ &- \frac{2}{mn}\sum_{i,j} k(x_i,y_j).\end{aligned} \tag{2}$$

For dataset comparison, P and Q are taken as the empirical distributions of two datasets D and D', and $\widehat{\mathrm{MMD}}^2(D,D')$ is used as their dissimilarity score. A common choice is the RBF kernel $k(x,y) = \exp\big(-\|x-y\|^2/(2\sigma^2)\big)$ with bandwidth σ selected via the median heuristic.

B. Task2Vec

Overview: Achille et al. [2] propose Task2Vec, a method for embedding machine learning tasks into a fixed-dimensional vector space such that geometric distances between embeddings reflect meaningful task similarity. The core idea is to use the diagonal of the Fisher Information Matrix (FIM) of a pretrained *probe network* — a fixed feature extractor briefly fine-tuned on the target task — as the task embedding. This captures the sensitivity of the probe's parameters to the task's data distribution, producing embeddings that are stable, comparable across tasks, and correlated with transfer learning performance.

Methodology: Given a dataset $D = (X,Y)$ and a probe network with parameters θ, the probe is fine-tuned on D by minimizing the task loss $\mathcal{L}(\theta;D)$. The Task2Vec embedding is the diagonal of the empirical Fisher Information Matrix:

$$\varphi(D) := \mathrm{diag}\big(\hat{F}(\theta)\big) \in \mathbb{R}^P, \tag{3}$$

where the empirical FIM diagonal is estimated as

$$\hat{F}_{ii}(\theta) = \frac{1}{N}\sum_{n=1}^{N}\left(\frac{\partial \log p(y_n \mid x_n,\theta)}{\partial \theta_i}\right)^2. \tag{4}$$

The resulting embedding $\varphi(D)$ encodes which parameters of the probe are most activated by dataset D. Task similarity is then measured via the cosine distance between embeddings:

$$d(D,D') = 1 - \frac{\varphi(D)^\top \varphi(D')}{\|\varphi(D)\|\cdot\|\varphi(D')\|}. \tag{5}$$

C. Dataset2Vec

Overview: Jomaa et al. [3] propose Dataset2Vec, a learned meta-feature extractor designed to replace hand-engineered dataset statistics in meta-learning pipelines. The primary motivation is that traditional engineered meta-features require expert domain knowledge and cannot handle datasets with varying schemas, while autoencoder-based alternatives are restricted to fixed-schema datasets. Dataset2Vec addresses both limitations by representing any tabular dataset as a hierarchical set — specifically as a set of predictor/target pairs — and processing it through a permutation-invariant DeepSet architecture [4]. To overcome the scarcity of meta-labeled data, the authors introduce an auxiliary meta-task called *dataset similarity learning*, which provides abundant self-supervised training signal by asking the model to predict whether two subsampled batches originate from the same dataset.

Methodology:

a) Architecture.: The meta-feature extractor is a three-module composition

$$\hat{\varphi} := h \circ g \circ f, \tag{6}$$

where f maps individual predictor/target pairs to embeddings, g aggregates those embeddings via pooling, and h maps the pooled representation to the final meta-feature vector. Each module is a stack of fully-connected layers with residual connections and ReLU activations. (Within this subsection, the symbols f, g, h refer exclusively to the Dataset2Vec sub-networks.)

b) Multi-fidelity batch sampling.: Meta-features are estimated from random multi-fidelity subsets. Given dataset D with N instances, M predictors, and T targets, a batch is drawn by sampling index subsets

$$N' \sim \{2^q \mid q \in [4,8]\}, \quad M' \sim [1,M], \quad T' \sim [1,T], \tag{7}$$

uniformly without replacement. The dataset-level meta-feature vector is estimated as the average of B batch-level estimates:

$$\hat{\varphi}(D) := \frac{1}{B}\sum_{b=1}^{B}\hat{\varphi}\big(\text{sample-batch}(D)\big). \tag{8}$$

c) Dataset similarity learning.: The auxiliary training task asks the model to determine whether two batches D_s and D'_s originate from the same dataset. The probabilistic similarity model is

$$\hat{i}(D_s,D'_s) := \exp\big(-\gamma\,\|\hat{\varphi}(D_s) - \hat{\varphi}(D'_s)\|\big), \tag{9}$$

and the training objective is a symmetric binary cross-entropy:

$$\begin{aligned}\min_{\hat{\varphi}} \; &-\frac{1}{|\mathcal{P}|}\sum_{(D_s,D'_s)\in\mathcal{P}} \log \hat{i}(D_s,D'_s) \\ &-\frac{1}{|\mathcal{W}|}\sum_{(D_s,D'_s)\in\mathcal{W}} \log\big(1-\hat{i}(D_s,D'_s)\big),\end{aligned} \tag{10}$$

where $\mathcal{P}$ and $\mathcal{W}$ are the sets of similar and dissimilar batch pairs, respectively.

D. Wasserstein Task Embedding

Overview: Wasserstein Task Embedding (WTE) *Liu et al. (2022)* is a model-agnostic task representation method for supervised classification that combines multidimensional scaling (MDS) with Wasserstein embedding. The main idea is to represent every task as a probability measure in an augmented space where the label information is first embedded into a Euclidean vector space and then concatenated with the input

samples. This makes it possible to approximate a hierarchical optimal transport dataset distance with a standard Euclidean distance between task vectors.

Compared with pairwise optimal transport methods such as OTDD, WTE is designed to be substantially faster when many tasks need to be compared repeatedly. The paper shows that the resulting task distances are strongly correlated with both forward transfer and catastrophic forgetting, while also remaining highly correlated with OTDD on benchmark task groups.

Methodology: Let a task be given by a supervised dataset $\tau = \{(x_n, y_n)\}_{n=1}^{N}$ with inputs $x_n \in \mathbb{R}^d$ and labels $y_n \in \mathcal{Y}$. WTE first defines a label-to-label distance matrix using the 2-Wasserstein distance between label distributions. Following the simplification used in the paper, each class-conditional label distribution ν_y is approximated by a Gaussian, so that the distance between labels can be written in closed form as the Bures–Wasserstein distance:

$$W_2^2(\nu_y, \nu_{y'}) = \|u_y - u_{y'}\|_2^2 + \mathrm{Tr}\Big(\Sigma_y + \Sigma_{y'} - 2(\Sigma_y^{1/2}\Sigma_{y'}\Sigma_y^{1/2})^{1/2}\Big), \tag{11}$$

where u_y and Σ_y denote the mean and covariance of the Gaussian associated with label y.

The resulting pairwise label distances are then embedded into a low-dimensional Euclidean space using multidimensional scaling. Denoting the MDS map by ψ, each label y is mapped to a vector $\psi(\nu_y) \in \mathbb{R}^l$, where l is chosen as a compromise between accuracy and computational cost. This gives the approximation

$$W_2^2(\nu_y, \nu_{y'}) \approx \|\psi(\nu_y) - \psi(\nu_{y'})\|_2^2. \tag{12}$$

After label embedding, each original sample-label pair is converted to an augmented vector

$$[x, \psi(\nu_y)] \in \mathbb{R}^{d+l}. \tag{13}$$

The original task is therefore transformed into an updated point cloud in the augmented space, and the task distance becomes

$$d_\tau\big((x, y), (x', y')\big) \approx \|[x, \psi(\nu_y)] - [x', \psi(\nu_{y'})]\|_2 . \tag{14}$$

In this representation, the label discrepancy is absorbed into the feature space, allowing the task comparison problem to be reduced to a standard Wasserstein embedding problem.

To obtain the final task vector, WTE applies the Wasserstein embedding framework with respect to a fixed reference measure μ_0. For each task distribution μ_i over the augmented samples, an optimal transport map T_i from μ_0 to μ_i is approximated, and the embedding is defined as

$$\Phi(\mu_i) = (T_i - \mathrm{id})\sqrt{p_0}, \tag{15}$$

or, in the discrete setting used in the implementation,

$$\Phi(X_i) = \frac{T_i - X_0}{\sqrt{N_0}}, \tag{16}$$

where X_0 denotes the reference sample set and N_0 is its size. The Euclidean distance between the embedded task vectors then approximates the 2-Wasserstein distance between tasks:

$$\|\Phi(\mu_i) - \Phi(\mu_j)\|_2 \approx W_2(\mu_i, \mu_j). \tag{17}$$

Algorithmically, WTE proceeds in three steps: (1) compute the label distance matrix, (2) embed labels with MDS, and (3) apply Wasserstein embedding to the augmented task distributions. The paper reports that this pipeline reduces the cost of task comparison from pairwise optimal transport over all task pairs to a single transport computation per task against the reference measure, which is the main source of the computational gain.

IV. Research Methodology

Our research methodology involved comprehensive implementation and experimental validation of each dataset embedding technique. We followed a systematic approach:

- **Library Design**: We built a unified PyTorch-compatible interface where each embedding method is implemented as a self-contained module with consistent `fit()` and `embed()` methods.
- **Algorithm Implementation**: Each method was implemented as a separate class following a common base interface, enabling straightforward benchmarking and extension.
- **Experimental Validation**: We conducted controlled experiments on standard meta-dataset benchmarks to evaluate embedding quality, dataset similarity retrieval accuracy, and downstream meta-learning performance.

A. Repository Structure

The repository is organized into three main layers. The `src/` directory contains reusable library components, the `benchmarks/` directory contains executable evaluation pipelines, and the `demo/` directory provides lightweight notebooks for qualitative inspection of the methods. In addition, the project includes unit tests for the main embedder modules and continuous-integration workflows for automated testing and documentation deployment.

B. Benchmarking Pipelines in the Repository

The public code base already contains concrete benchmark logic for three practical use cases of dataset embeddings.

a) Dataset2Vec warm-start HPO benchmark.: The benchmark in `benchmarks/dataset2vec_hpo/` constructs a small surrogate hyperparameter-optimization environment over tabular classification tasks. It combines four standard datasets (`iris`, `wine`, `breast_cancer`, `digits`) with four synthetic datasets and evaluates a grid of 108 MLP hyperparameter configurations. For each held-out target dataset, three strategies are compared: cold-start random search, warm start from engineered statistical meta-features, and warm start from Dataset2Vec embeddings. Performance is summarized with the ADTM (Average Distance to the Minimum) metric over multiple random seeds.

b) Task2Vec transfer-selection benchmark.: The notebook `benchmarks/pretrain_benchmark/apply_benchmark.ipynb` uses Task2Vec embeddings to retrieve the nearest source dataset for a given target image task. The benchmark

operates on datasets such as `mnist`, `cifar10`, `cifar100`, `letters`, and `kmnist`, computes embeddings with a pretrained `ResNet-18` probe network, and then compares the retrieval-based recommendation against logged pretrain-to-task transfer results. The same notebook also defines random and large pretraining baselines.

c) Wasserstein transfer-selection benchmark.: The notebook `benchmarks/pretrain_benchmark/wasserstein/benchmark_final_experiment.ipynb` implements a similar transfer-selection experiment using Wasserstein-based dataset distances. It first computes class-wise Bures–Wasserstein distances, derives dataset-level distances from the resulting embeddings, and then recommends the closest source dataset for each target task. When transfer logs are available, the notebook compares these recommendations with observed downstream accuracies.

d) Status of MMD in the repository.: MMD is retained in this report as an important classical baseline and as part of the conceptual comparison framework. At the same time, the current public repository places its strongest executable emphasis on Dataset2Vec, Task2Vec, and Wasserstein components, including demos and benchmark notebooks. Packaging MMD as a first-class module would be a natural next step for future development.

C. Demonstration Notebooks

The repository also includes method-specific notebooks intended for qualitative validation and onboarding.

a) Task2Vec demo.: The notebook `demo/task2vec/simple_example.ipynb` embeds a small collection of image datasets with a pretrained `ResNet-18` probe and visualizes the resulting task distances with a clustered similarity matrix.

b) Dataset2Vec demo.: The notebook `demo/dataset2vec/simple_example.ipynb` constructs a balanced synthetic tabular meta-dataset, trains a Dataset2Vec encoder, and studies the resulting embeddings using PCA/t-SNE projections together with nearest-centroid classification in embedding space.

c) Wasserstein demo.: The notebook `demo/wasserstein/simple_example1 (1).ipynb` demonstrates the full Wasserstein pipeline: computation of pairwise class distances, MDS-based label embeddings, augmentation of the feature space, and construction of reference-based task embeddings.

V. Analysis and Interpretation

At the current stage, the repository is strongest not in frozen numeric tables but in reproducible experimental pipelines. The available scripts already support three central empirical questions: (i) whether embeddings recover meaningful nearest-neighbor datasets, (ii) whether those neighbors improve source-task selection for transfer learning, and (iii) whether learned meta-features reduce the burn-in cost of hyperparameter search.

The benchmark organization also reveals a practical division of roles between the methods. Dataset2Vec is the main tabular meta-feature learner and is currently tied to the most complete HPO benchmark. Task2Vec is used primarily for image-task retrieval through probe-network Fisher embeddings, while Wasserstein embedding emphasizes geometry-aware dataset comparison through optimal transport. This is a useful point to state explicitly in the report, because it explains why the benchmarking suite is method specific rather than perfectly uniform across all algorithms.

When the final experimental runs are completed, this section can be extended with the exact artifacts already anticipated by the repository: ADTM curves for HPO, dataset-to-dataset distance heatmaps, and nearest-neighbor transfer recommendation tables.

VI. Conclusions and Recommendations

`DataMetaMap` provides a growing toolkit for dataset representation and comparison. In its current public form, the repository combines packaged Dataset2Vec and Wasserstein embedders, a Task2Vec subpackage, theoretical coverage of MMD as a classical baseline, and benchmark notebooks for transfer and meta-learning scenarios.

The existing benchmark suite is designed to test the hypothesis that learned embeddings capture richer dataset structure than purely distributional baselines, especially when schemas vary across tasks. We encourage further work on standardizing the benchmarking interface across all methods, extending the released result tables with fully reproducible numeric comparisons, and turning MMD into a packaged executable component of the library.

Appendix A
Demo Experiments

Our demo code is available at the GitHub repository. The experiments are divided into the following groups:

- **Task2Vec Similarity Demo**: A notebook that embeds `mnist`, `cifar10`, `cifar100`, and `letters` with a pretrained `ResNet-18` probe and visualizes the resulting task-distance matrix.
- **Dataset2Vec Training and Visualization Demo**: A synthetic tabular meta-dataset is generated, a Dataset2Vec model is trained on balanced train/validation/test splits, and the learned representations are explored with PCA/t-SNE projections and nearest-centroid classification.
- **Wasserstein Embedding Demo**: A compact notebook computes class-wise Bures–Wasserstein distances, label embeddings, and final task embeddings for small image-dataset collections.
- **Hyperparameter-Optimization Benchmark**: An end-to-end script compares random search, warm start from engineered meta-features, and warm start from Dataset2Vec embeddings under the ADTM metric.
- **Transfer-Based Benchmark Notebooks**: Separate notebooks evaluate Task2Vec and Wasserstein retrieval as heuristics for choosing a source pretraining dataset before downstream fine-tuning.

REFERENCES

[1] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, "A kernel two-sample test," *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012.

[2] A. Achille, M. Lam, R. Tewari, A. Ravichandran, S. Maji, C. Fowlkes, S. Soatto, and P. Perona, "Task2vec: Task embedding for meta-learning," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[3] H. S. Jomaa, L. Schmidt-Thieme, and J. Grabocka, "Dataset2vec: Learning dataset meta-features," *Data Mining and Knowledge Discovery*, 2021, arXiv:1905.11063.

[4] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola, "Deep sets," in *Advances in Neural Information Processing Systems*, 2017.